YOJANAMITRA

Student Opportunity Discovery and Application Support System

MINI PROJECT REPORT

Submitted in partial fulfilment of the Semester-I project requirements, Academic Year 2026-27

Submitted by

[Student Name] | [Roll Number] | [Class / Division]

[Group member names and roll numbers, if applicable]

Project Guide: [Guide Name]

[Department / Programme]

[College / Institute Name]

Academic Year 2026-27

CERTIFICATE

This is to certify that this report entitled “YojanaMitra: Student Opportunity Discovery and Application Support System” describes project work carried out by the student(s) named on the title page during Semester-I of Academic Year 2026-27 under the guidance of the undersigned. The work is submitted in partial fulfilment of the academic requirements of [College / University Name].

Complete the student, guide, department, and institute fields accurately and obtain the required signatures before submission.

Project Guide	Head of Department	Principal
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Place: _____ Date: _____

DECLARATION

I/We declare that this report describes the YojanaMitra mini project completed as academic work for Semester-I, 2026-27. Implementation details and test results are based on the project source and the verification run described in Chapter 4. Technical references are acknowledged. I/We understand that the fields for student, guide, and institution must be completed before submission. Scheme details and deadlines must be checked on the relevant official portal before a real application.

Student name(s): _____

Roll number(s): _____

Signature(s): _____ Date: _____

ABSTRACT

YojanaMitra is a web application that helps students discover scholarships, public opportunities, jobs, and internships; understand recorded eligibility conditions; and organize applications. A student account stores a profile with details such as course, category, annual family income, domicile, marks, and language. The server compares that profile with structured scheme rules and returns a status with reasons. Missing inputs and extra manual conditions remain “Maybe” instead of being treated as a pass.

The system uses a React and TypeScript client built with Vite, an Express and TypeScript API, Prisma ORM, and SQLite. Authenticated records include profile, bookmarks, applications, document checklist entries, and assistant sessions. The app has opportunity discovery, job tracking, administrator review, and an assistant with separate YojanaMitra and general modes. Optional voice input and speech output depend on browser capability, microphone permission, and configured transcription providers.

The report explains the system with architecture, data-flow, database, user-flow, and eligibility diagrams and includes screenshots from the local application. At the evaluation run, nine server tests passed; server/client TypeScript checks and production builds completed. These results confirm code paths exercised by the checks, not the accuracy of every seeded scheme or the availability of external AI providers. The official portal remains the authority for current eligibility, documents, benefits, and deadlines.

Keywords: student support, scholarships, eligibility, application tracking, conversational assistant, React, Express, SQLite.

CONTENTS

Abstract iv

List of Abbreviations vi

List of Figures and Screenshots vi

Chapter 1 Introduction 1

Chapter 2 Literature Survey 3

Chapter 3 System Development 5

Chapter 4 Performance Evaluation 14

Chapter 5 Conclusion and Future Scope 16

References 17

Acknowledgement 18

LIST OF ABBREVIATIONS

Term	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
DB	Database
DFD	Data Flow Diagram
GR	Government Resolution
JWT	JSON Web Token
NSP	National Scholarship Portal
ORM	Object-Relational Mapping
UI	User Interface
URL	Uniform Resource Locator

LIST OF FIGURES AND SCREENSHOTS

3.1 System architecture

3.2 DFD Level 0: system context

3.3 DFD Level 1: major processes

3.4 DFD Level 2: eligibility assessment

3.5 Student and administrator use cases

3.6 Core database relationships

3.7 Eligibility result logic

3.8 Assistant and voice request flow

3.9 Student profile screen

3.10 Assistant screen

CHAPTER 1 INTRODUCTION

1.1 Background

Students often find scholarships, education support, internships, and recruitment notices across separate portals. Each opportunity can have different eligibility conditions, document lists, dates, and application steps. A student must compare these requirements with personal details, prepare certificates, and remember multiple application statuses. Scattered information increases the chance of overlooking a suitable opportunity or relying on an old summary.

YojanaMitra organizes discovery and preparation in one authenticated workspace. Students save a profile, search a catalog, compare their details with recorded rules, save opportunities, track applications, and open official sources. An assistant answers questions using relevant stored records in its guidance mode, while a separate general mode does not receive profile or catalog context.

1.2 Problem statement

The project addresses the repetitive work of finding opportunities and managing application preparation. Students need an accessible way to compare stated rules with their profile, understand what remains unknown, retain their details between visits, and track next steps. The system must not turn incomplete or unreviewed information into a promise of eligibility or an official deadline.

1.3 Aim and objectives

Objective	Project support
Discovery	Search scholarships, government/private jobs, and internships.
Personal comparison	Apply explicit rule checks and explain outcomes.
Persistence	Save profile, bookmarks, documents, applications, and chat sessions per account.
Preparation	Provide checklists, source links, and a student-managed tracker.
Conversational support	Provide grounded guidance and separate general chat, with optional voice.
Review	Track sources and require administrator verification for publication.

1.4 Scope and boundaries

Implemented functions include accounts, profiles, opportunity search, rule-based screening, bookmarks, scheme/job application tracking, document checklist, chat history, voice controls, and administrator review. The app does not submit forms to external portals, issue certificates, verify identity with a government system, guarantee awards, or replace an official notification. External AI depends on provider configuration, quota, network, and model availability. Stored opportunity data requires human review for the active cycle.

1.5 Intended users

Actor	Tasks
Student	Maintain profile, discover and compare records, prepare documents, track applications, ask questions.
Administrator	Review scheme sources/rules, knowledge excerpts, and draft job notices.
Official portal	Authoritative source for current eligibility, documents, dates, and application instructions.

1.6 Report organization

Chapter 2 reviews information sources, manual workflows, rule-based screening, assistants, and the project technology. Chapter 3 explains requirements, architecture, data flows, schema, interface, processing, and security. Chapter 4 reports tests and their limits. Chapter 5 concludes and lists practical improvements.

CHAPTER 2 LITERATURE SURVEY

2.1 Opportunity information sources

Scholarship and recruitment details are published by government departments, institutions, examinations, and recruiting organizations. Their official portals and notices are authoritative for eligibility, documents, benefit, and deadlines. An aggregator can improve navigation, but it should preserve a source link and should not present a summary as a replacement for the publishing authority.

Students may also encounter private aggregators, social posts, coaching pages, and forwarded messages. These can help surface an opportunity but may be incomplete or stale. YojanaMitra records source URLs, review state, and dates and asks users to verify current details on the official portal. Deadlines and rules can change each cycle.

2.2 Manual workflow limitations

A manual process asks students to search each portal, maintain a calendar, save documents separately, and remember whether an application is planned, submitted, or under review. It is possible but repetitive. Comparing income ceilings, course types, domicile, marks, and category across notices also takes attention.

The project consolidates these activities. A saved profile reduces repeated entry; an explainable comparison surfaces known matches and missing data; bookmarks preserve options; and the tracker stores status and notes. These are organization tools. A public authority still decides the final application outcome.

2.3 Rule-based screening

A deterministic engine compares explicit fields in structured data. It can test annual income against a threshold, category against an allowed list, or marks against a minimum and give a reason for each result. Its trustworthiness depends on accurate profile values and source rules, and it cannot infer conditions that have not been modeled.

YojanaMitra reports “Not eligible” when a modeled requirement fails, “Maybe” when required data is missing or a manual condition remains, and “Eligible” only when the modeled conditions pass. Empty rule sets do not produce a positive result. Even a pass includes a reminder to check the official portal.

2.4 Conversational assistants and retrieval

A conversational assistant accepts ordinary-language questions. In grounded mode, the server retrieves relevant scheme, job, or knowledge records and supplies that context to a configured model. Source links can accompany the reply. The model instructions discourage invented dates and eligibility rules. When providers fail, the system can show stored guidance or an offline response.

General mode deliberately omits profile information and retrieved scheme data. Voice support uses browser speech functions where available or a configured transcription provider for recorded audio. Provider quota, model availability, browser implementation, and microphone permissions affect results. The assistant remains guidance, not an official decision.

2.5 Technology comparison

Technology	Role	Project fit
React 19	Interactive client components	Reusable stateful interface.
TypeScript 5	Static typing	Checks API and rule data shapes.
Vite 7	Development and bundling	Client development and production build.
Node.js / Express 5	HTTP API	Middleware and application routes.
Prisma 6 / SQLite	Database access	Typed model access with compact relational persistence.
Zod	Runtime validation	Rejects invalid requests at server boundaries.

2.6 Security and privacy principles

The implementation uses authentication, Argon2 password hashing, per-user queries, Zod validation, CSRF checks for mutations, Helmet security headers, and rate limits for login and AI chat. Authentication uses an HTTP-only cookie. The API sends no-store headers outside tests. Guidance mode passes a limited masked profile summary; general mode receives no profile or local scheme context. Account export and deletion routes are implemented.

These controls reduce common risks but are not an independent penetration test. A deployment still needs a strong secret, HTTPS, correct allowed origins, database backup and restore, careful provider-key handling, dependency updates, and a human source-review process.

2.7 Summary

The design combines discovery and personal task management while preserving source links and uncertainty. Rule checks provide transparent reasons; the assistant helps with natural-language questions. Ongoing human data review remains essential because code validation cannot prove that a transcribed official rule is correct.

CHAPTER 3 SYSTEM DEVELOPMENT

3.1 Development structure

The system is separated into a browser client, an API, shared route constants, and a database. The browser gathers input and displays status; the server validates requests and applies security/business rules; Prisma persists data in SQLite. Sensitive account access and eligibility evaluation occur server-side rather than relying on transient page state.

3.2 Functional requirements

ID	Requirement	Expected behaviour
FR1	Account	Register, sign in, inspect session, sign out.
FR2	Profile	Save student data and retrieve it after refresh/sign-in.
FR3	Discovery	Search opportunities and show source, review notice, deadline, and match reasons.
FR4	Eligibility	Return three-state status with reasons from structured criteria.
FR5	Tracking	Manage bookmarks, application statuses, and notes.
FR6	Documents	Maintain checklist items and direct users to authoritative instructions.
FR7	Jobs	View published notices and track/bookmark job applications.
FR8	Assistant	Choose grounded or general mode and retain user-owned chat sessions.
FR9	Voice	Accept speech and optionally speak replies where supported.
FR10	Administration	Edit scheme/knowledge data and review/publish job notices.
FR11	Data rights	Export account records or delete account after password confirmation.

3.3 Quality requirements

Area	Design / evidence
Usability	Workspace navigation, form labels, status explanations, EN/HI/MR language labels.
Reliability	Database persistence, request validation, missing data remains uncertain.
Security	Argon2, signed session cookie, CSRF, Helmet, rate limits, scoped queries.
Privacy	General AI mode omits profile and retrieval context; guidance sends limited masked summary.
Maintainability	Separate workspaces, shared route constants, Prisma schema, deterministic tested logic.
Accessibility	Semantic controls/labels/live regions exist; a full assistive-technology audit remains future work.
Portability	Client/server can run independently and be built for deployment.

3.4 System architecture

Figure 3.1 shows the browser, React client, Express API, SQLite database, and optional AI providers. Requests pass through server authentication and validation. Prisma reads or writes application data. AI requests reach a configured external or local provider; browser speech facilities support optional input/output.

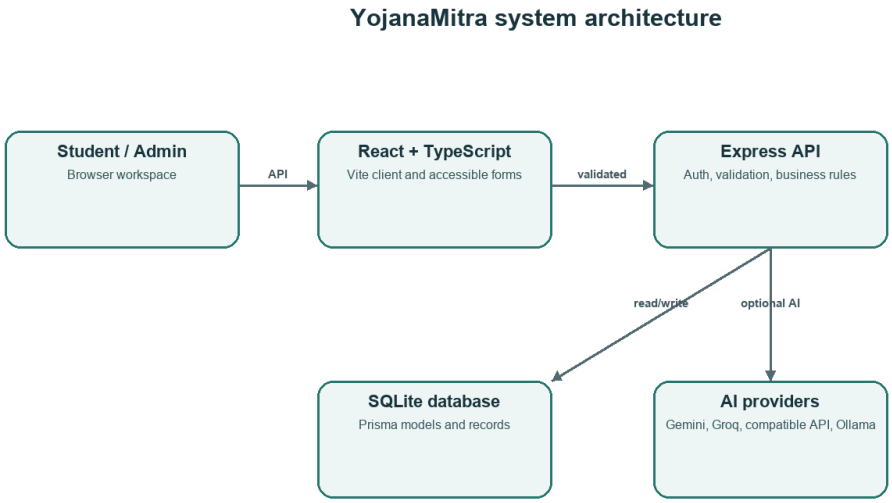


Figure 3.1 Client, API, database, and AI provider architecture

3.5 Data-flow diagrams

3.5.1 Level 0 context

Figure 3.2 treats the application as one process. Students provide profile details, questions, and status updates; administrators provide reviewed records. The system returns matches, explanations, and tracker information. Official portals remain outside the system boundary and provide final confirmation.

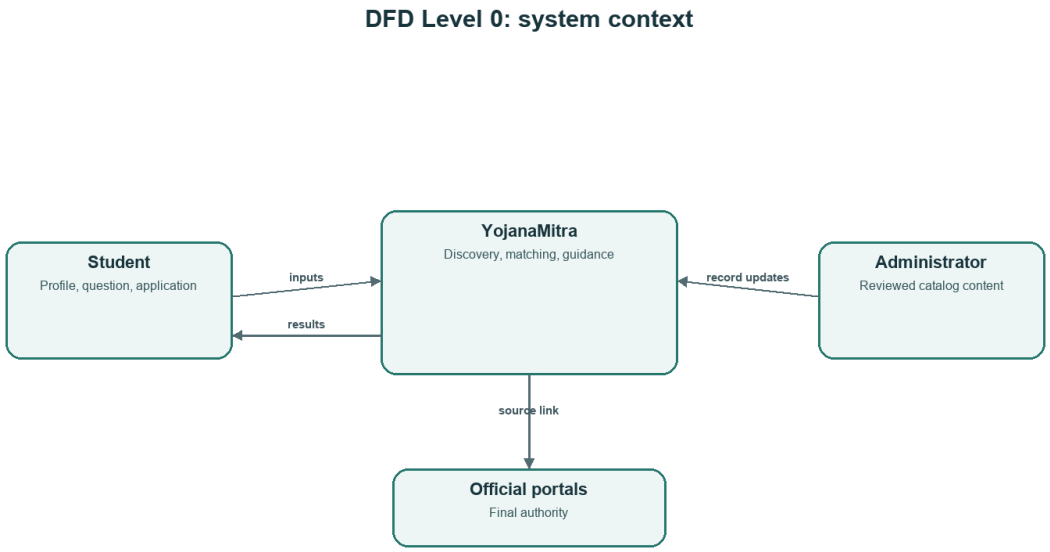


Figure 3.2 DFD Level 0: actors and system boundary

3.5.2 Level 1 processes

Figure 3.3 separates profile/account, catalog, eligibility, tracking, assistant, and administration. These processes use the persistent store, while private records are scoped to the signed-in account. A student may progress from profile and search to eligibility and then save or track an opportunity.

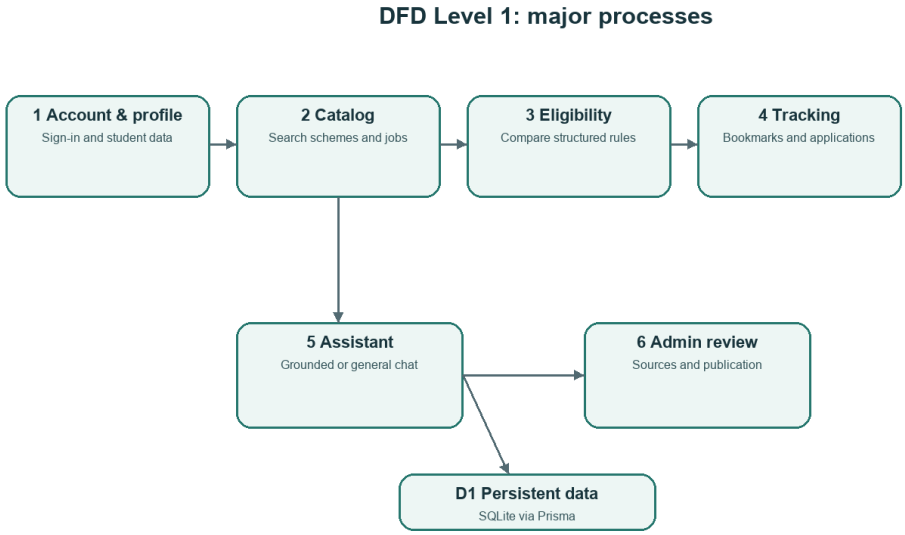


Figure 3.3 DFD Level 1: main processes and data store

3.5.3 Level 2 eligibility

Figure 3.4 expands eligibility calculation. The system combines a profile with a scheme's rules, checks supported criteria, and keeps missing values unknown. A failed requirement produces

Not eligible; missing information or an unmodeled condition produces Maybe; a complete pass produces Eligible with reasons and a reminder.

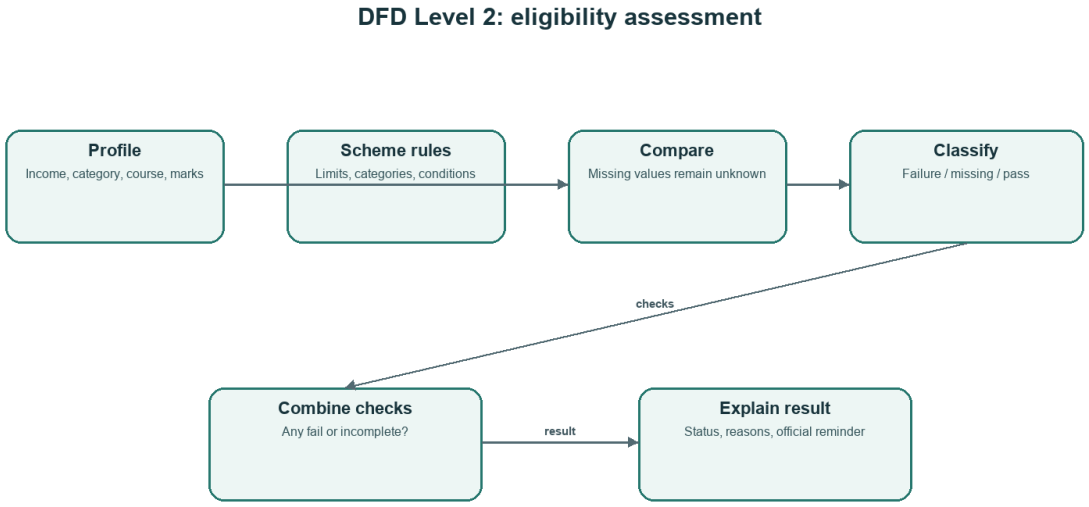


Figure 3.4 DFD Level 2: profile-to-result flow

3.6 User journeys

Figure 3.5 shows student and administrator actions. Students search and inspect records, manage their profile and application work, and ask questions. Administrators review source-linked content. A student must open the current notice and submit applications independently.

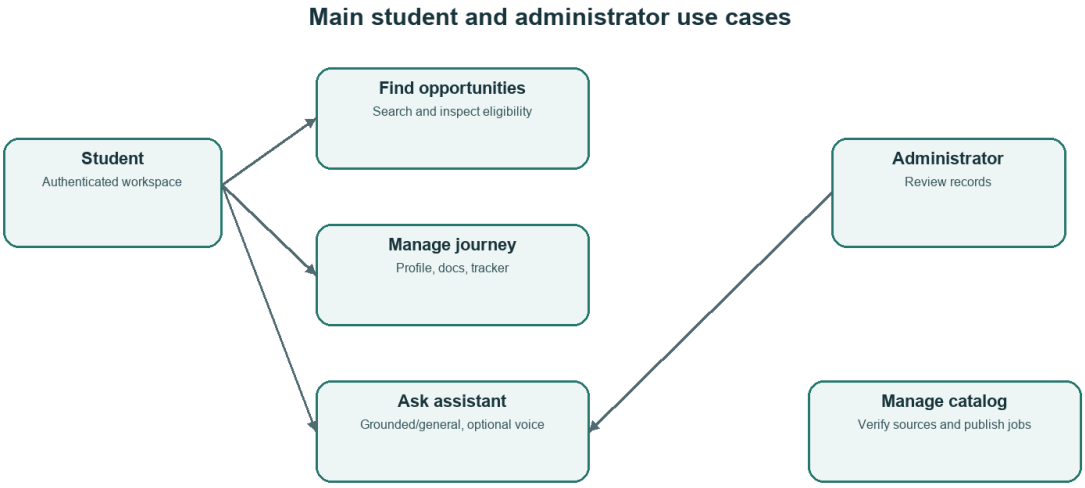


Figure 3.5 Student and administrator use cases

3.7 Database model

User is the root account entity. Profile is optional and has a unique userId. Scheme and JobPost contain different kinds of opportunities. Application, Bookmark, JobApplication, and JobBookmark link accounts to opportunities with uniqueness constraints that prevent duplicate tracking. ChatSession belongs to a user and contains messages with source references. Account deletion cascades through dependent data.

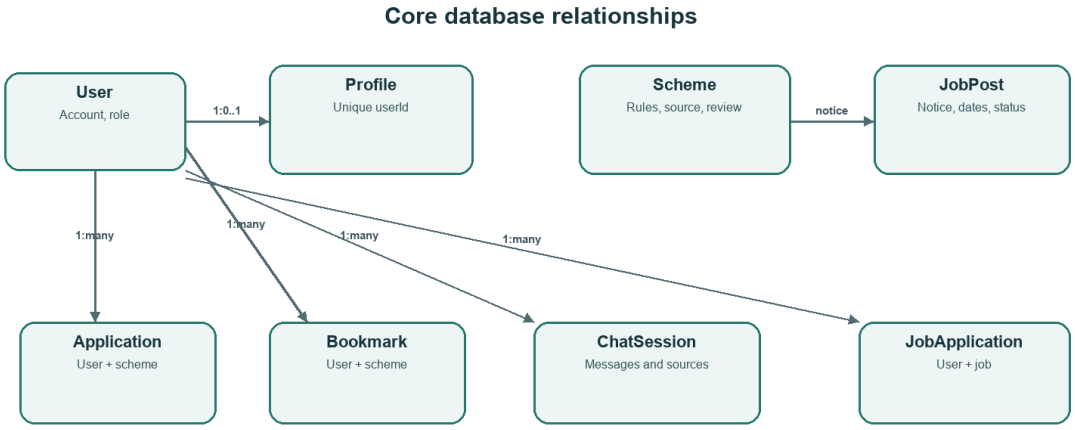


Figure 3.6 Simplified entity relationship model

Entity	Purpose / key fields
User	Account email, role, password hash.
Profile	Category, income, location/domicile, course, marks, language.
Scheme	Rules, benefits, documents, deadline, source, verification dates.
Application / Bookmark	User-to-scheme tracking and saved opportunity.
JobPost	Notice, qualifications, dates, source evidence, review state.
ChatSession / Message	Owner, title, text, role, sources, timestamps.
KnowledgeChunk	Topic, language, source URL/name, review state.
JobApplication / JobBookmark	User-to-job tracking and saved opportunity.

3.8 API design

Express routes group authentication, profile, schemes, bookmarks, applications, documents, jobs, chat sessions, AI, and administration. Shared route constants reduce client/server path mismatch. Authentication middleware reads the session cookie. Mutations compare a CSRF header with a cookie. Administrator routes require ADMIN role. Zod validates input before database writes.

3.9 Eligibility engine

The TypeScript engine supports income ceilings/floors, category, course keywords, course type, minimum marks, state, gender, domicile, disability, age range, and manual checks. Missing required values create reasons. A failed check yields Not eligible; uncertainty yields Maybe; and only complete modeled success yields Eligible. The job evaluator uses the notification cutoff

date and recorded category relaxation, while incomplete age or qualification data remains uncertain.

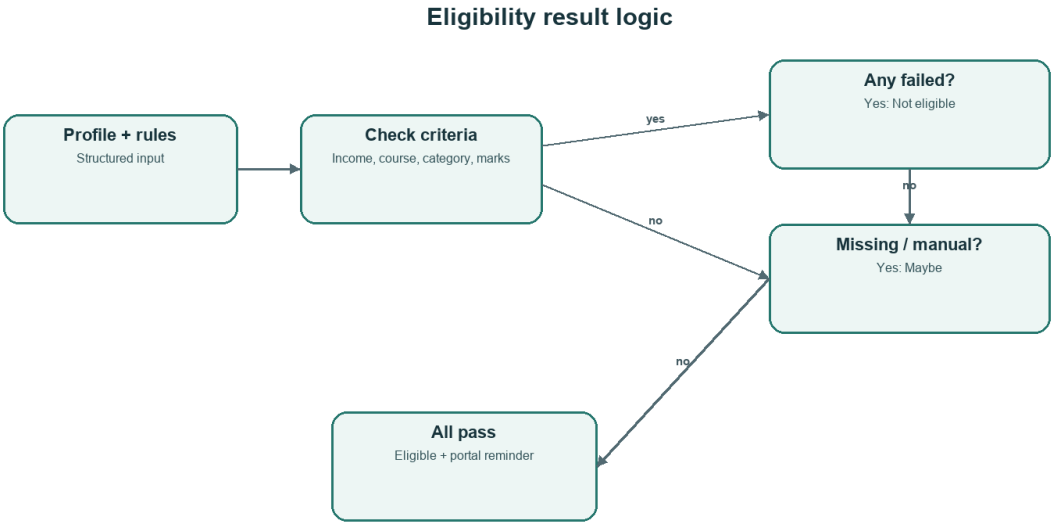


Figure 3.7 Decision logic for the eligibility outcomes

For an income ceiling, an amount equal to the limit passes the criterion; one rupee above fails it. Missing income is not treated as zero. Manual checks prevent a complete-pass result until the student checks the official condition. These behaviours are covered by automated server tests.

3.10 Assistant and voice flow

Figure 3.8 shows typed or spoken input, the chat API, mode-specific context, and reply. Guidance mode retrieves relevant local records and a limited profile summary. General mode skips profile and catalog retrieval. The server stores sessions and source references. If providers are unavailable, it returns stored guidance where possible or an explicit offline response.

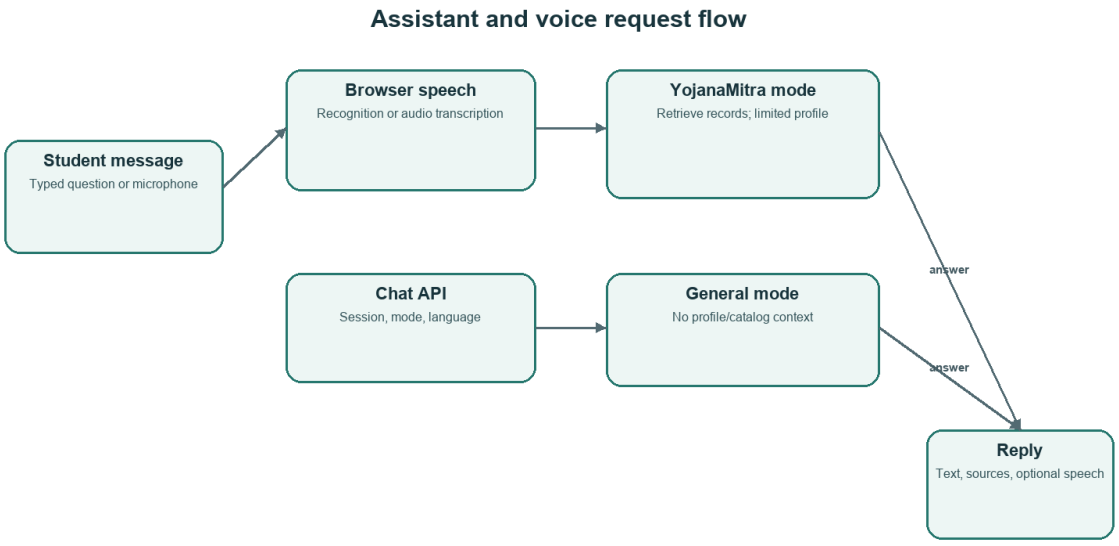


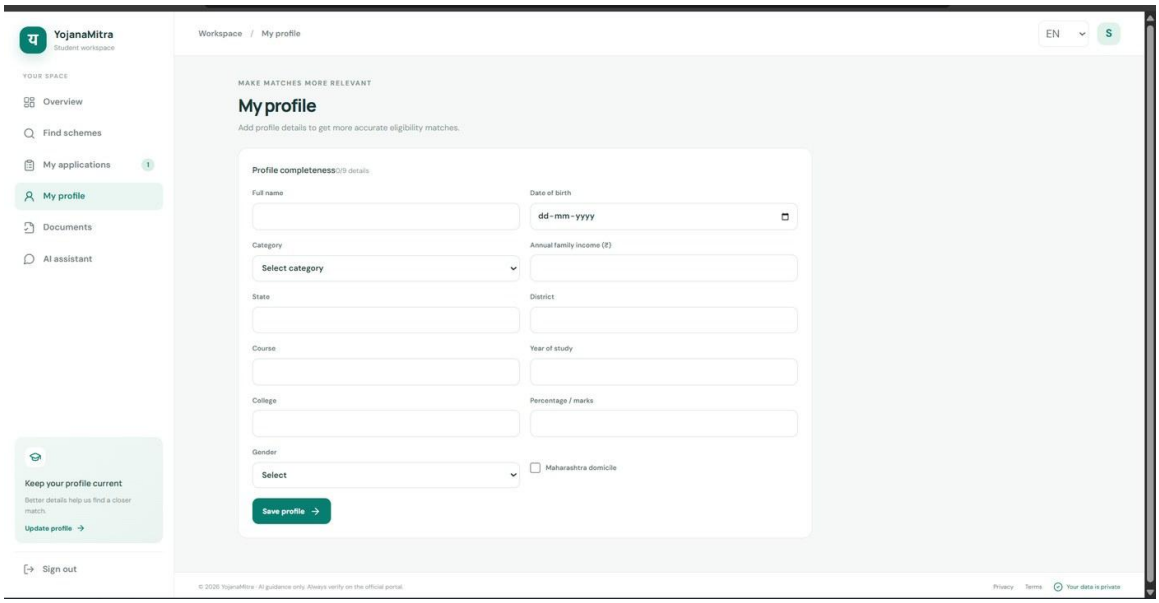
Figure 3.8 Assistant request path and optional voice processing

Voice input uses browser speech recognition when available; otherwise the client can record audio for server-side transcription through configured providers. Browser speech synthesis reads responses and supports speed, pause/resume, stop, and hands-free options. Permission or provider failure should leave typed input usable.

3.11 User interface

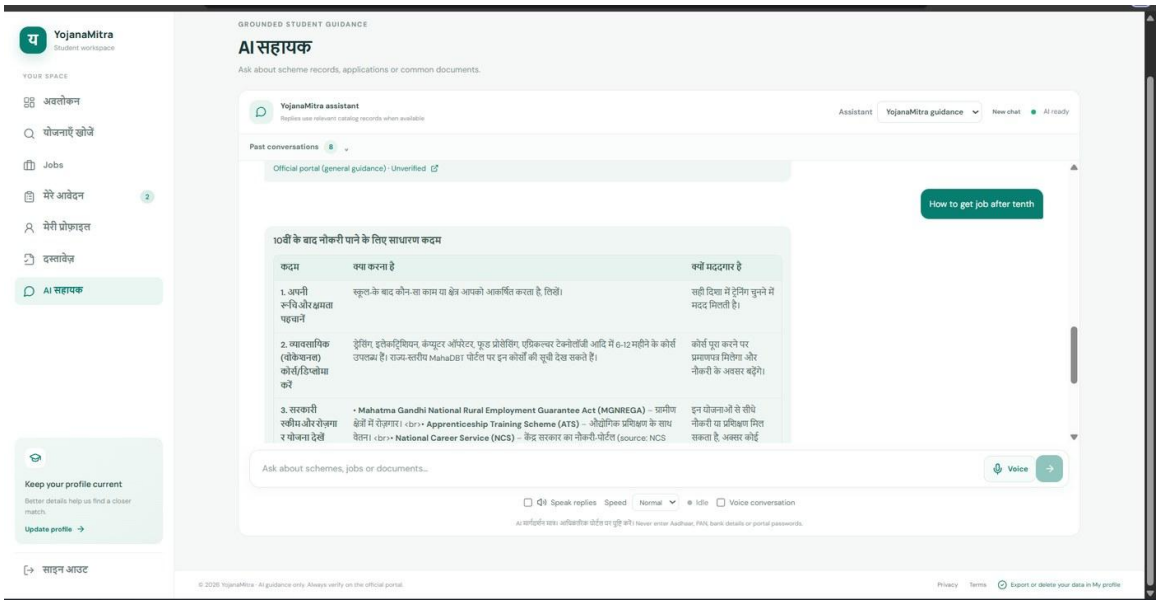
The workspace navigation includes overview, schemes, jobs, applications, profile, documents, and AI assistant. Opportunity cards show benefits text, a deadline value, last-verified state, eligibility result and reason, official source, and actions. “Check official site” and review notices distinguish unverified catalog data from official confirmation.

Screenshot 3.9 shows the profile form. Saving calls the authenticated profile endpoint, which stores the values in the database. On browser refresh the app requests the profile again from the server, so data persists while the account remains signed in; it is not only held in React memory.



Screenshot 3.9 Student profile page captured from the local YojanaMitra app.

Screenshot 3.10 shows assistant mode selection, conversation history, message area, microphone input, reply speech controls, speed, and voice conversation. The page presents a reminder to verify guidance on the official portal and avoid entering sensitive identity or banking information.



Screenshot 3.10 AI assistant screen captured from the local YojanaMitra app.

3.12 Source verification and administration

Scheme records store structured rules/documents, official and source URLs, verification state/date, and a notice. Changing source or rule content clears prior verification. Server checks require an exact verification status, real date, complete non-placeholder fields, a specific official

page, and no unresolved conflict in the notes. This prevents accidental verification but cannot judge whether a person copied a rule accurately.

Job records store the official notification, source details, qualifications, age rules, fees, dates, documents, and selection process. New jobs begin as drafts. Publishing requires an official notification URL and administrator verification. Stale non-evergreen published notices can be moved back to draft for review.

3.13 Security and privacy

Control	Implementation	Purpose
Passwords	Argon2 hash; strong password validation.	Avoid storing raw password.
Session	Signed JWT in HTTP-only cookie; seven-day expiry; SameSite=Lax.	Keep session token out of browser scripts.
CSRF	Random cookie/header equality on mutations.	Protect state-changing routes.
Validation	Zod schemas and JSON/audio limits.	Reject invalid or oversized inputs.
Access scope	User identity used in private record queries.	Limit data to account owner.
Browser protections	Helmet, microphone policy, API no-store headers.	Reduce common exposure.
Rate limits	Login/AI limits and failed-login tracking.	Slow guessing and overuse.
AI minimization	General mode gets no profile; guidance sends a limited masked summary.	Reduce provider data exposure.

3.14 Design trade-offs

SQLite simplifies local setup and is appropriate for an academic demonstration. A public multi-user deployment may require a managed database and tested backups. Deterministic rules are explainable but cannot capture every exception in a long notification. AI accepts natural questions but varies with provider behaviour and is therefore not used as the eligibility authority.

CHAPTER 4 PERFORMANCE EVALUATION

4.1 Evaluation method

Evaluation used automated server tests, project-wide TypeScript checks, and production builds. These checks assess internal code behaviour and buildability. They do not constitute a large-scale load benchmark, external security certification, or factual review of all opportunity records.

4.2 Environment and commands

Checks were run in the project workspace on 16 October 2026 with installed Node.js/npm dependencies. Commands were `npm test -w server`, `npm run typecheck`, and `npm run build`. The test command runs Node tests through tsx; typechecking covers server and client; the build compiles the server and creates the client Vite bundle.

4.3 Results

Check	Observed outcome	Interpretation
Server tests	9 passed; 0 failed.	Covered eligibility, jobs, search normalization, and tracker verification.
Typecheck	Completed for server and client.	No TypeScript errors reported in this run.
Production build	Server compilation and client Vite build completed.	Both workspaces built locally.
Screenshots	Profile and assistant screens captured.	Documents the UI; not a cross-browser usability audit.

4.4 Boundary cases

Scenario	Expected behavior	Reason
Income equals ceiling	Criterion passes.	Exact boundary is inclusive.
Income one rupee over	Criterion fails.	Ceiling is enforced.
Required profile value missing	Maybe with reason.	Unknown is not a pass.
Empty/unverified rule object	Maybe, with portal reminder.	Blank rule data cannot assert eligibility.
Manual official condition remains	Maybe.	Unmodeled exception remains unresolved.
Job age with cutoff/relaxation	Calculate using notice cutoff and category value.	Uses date stated by recruiting authority.

4.5 Test coverage and gaps

Area	Current automated coverage / gap
Eligibility	Unverified rules, income boundary, manual criteria.
Jobs	Common job-question recognition and cutoff-based age relaxations.
Search	Normalization of common scholarship queries.
Tracker verification	Exact Yes, real date, complete data, specific source, conflict notes.
Not in nine server tests	Browser end-to-end, microphone permissions, external provider availability, penetration tests, load, factual audit of seeded records.

4.6 Functional walkthrough

A student can register, sign in, complete and save a profile, search opportunities, inspect a source and eligibility explanation, bookmark or track a record, manage document notes, and ask the assistant. The assistant selector permits choosing general mode separately. Voice tests require microphone permission and compatible browser support; typed input remains the fallback.

4.7 Security review

Source inspection confirms authentication middleware, administrator role checks, password hashing, CSRF comparison for mutations, request validation, browser security headers, rate limits, and user-scoped queries. General mode skips profile and catalog context. These code-level controls have not been independently penetration-tested; production secret management, HTTPS, backups, and access operations still require attention.

4.8 Performance interpretation

No response-time or concurrency benchmark is claimed. AI response time varies with provider latency, quota, network, and model availability. SQLite suits the local demonstration but needs evaluation before scaling. The uncertainty statuses and source notices are intentional because an unknown rule should not be presented as a confirmed answer.

4.9 Limitations

Limitation	Effect	Follow-up
Seed accuracy not proven by code tests	Rules may be incomplete or stale.	Review each record on the current official page/GR.
Cycle dates change	Old deadline may be invalid.	Keep “Check official site” until confirmed.
AI provider dependency	Quota/network failure affects answers.	Monitor providers and keep offline/text fallback.
Browser voice variation	Microphone or recognition may fail.	Keep typed entry and clear status.
Not every clause is modeled	Exceptions require human review.	Use manual checks and Maybe.
No external load/security audit	Production readiness not established.	Test operations, security, accessibility, and scale before public launch.

CHAPTER 5 CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

YojanaMitra brings opportunity discovery, profile-based screening, document preparation, application tracking, and conversational help into one web application. Its key design principle is to expose known conditions and missing information while keeping official portals as the final source. Three-state results communicate failures and incomplete checks rather than overstating certainty.

The application has a React/TypeScript client, Express/TypeScript API, and Prisma/SQLite data layer. Account-associated data is persisted server-side. The project provides scheme and job workflows, reviewed-source mechanisms, separate grounded and general AI modes, and optional voice controls. In the recorded evaluation, nine server tests passed and project typechecking and builds completed.

The implementation is suitable as an academic project and demonstration. Its usefulness depends on accurate current records and sound deployment operations. Automated checks verify only the conditions they exercise. Students should confirm current eligibility, documents, benefits, and application dates on official portals.

5.2 Future enhancements

Priority	Enhancement	Benefit
1	Human source audit for every scheme/job.	Improves factual reliability and verified dates.
2	Review reminders and due/expired dashboard.	Makes maintenance repeatable.
3	Browser end-to-end and accessibility tests.	Covers real navigation, assistive technology, permissions.
4	Rule-type boundary tests.	Reduces regressions.
5	Production migrations and backup monitoring.	Improves deployed data resilience.
6	Consent-based saved-opportunity reminders.	Helps users follow deadlines.
7	Multilingual AI grounding/citation evaluation.	Measures quality in supported languages.
8	Realistic concurrency and managed database evaluation.	Creates scaling evidence.

5.3 Student reminder

Use YojanaMitra to organize discovery and preparation. Confirm eligibility, documents, benefit amount, dates, and instructions on the official portal. Never enter Aadhaar, PAN, bank details, or portal passwords into the assistant.

REFERENCES

[1] YojanaMitra source code and Prisma schema: client/src/App.tsx; server/src/index.ts; server/src/eligibility.ts; server/src/jobs-eligibility.ts; server/src/tracker-verification.ts; server/prisma/schema.prisma. Inspected 16 October 2026.

[2] React Reference. https://react.dev/reference/react (accessed 16 October 2026).

[3] Vite Getting Started. https://vite.dev/guide/ (accessed 16 October 2026).

[4] TypeScript Handbook. https://www.typescriptlang.org/docs/handbook/ (accessed 16 October 2026).

[5] Express: Using middleware. https://expressjs.com/guide/using-middleware/ (accessed 16 October 2026).

[6] Node.js Introduction. https://nodejs.org/learn/getting-started/introduction-to-nodejs (accessed 16 October 2026).

[7] Prisma ORM v6 SQLite connector. https://docs.prisma.io/docs/orm/v6/overview/databases/sqlite (accessed 16 October 2026).

[8] SQLite About page. https://www.sqlite.org/about.html (accessed 16 October 2026).

[9] MahaDBT official portal: https://mahadbt.maharashtra.gov.in/; National Scholarship Portal: https://scholarships.gov.in/. Verify specific current scheme notices directly; home pages alone do not validate individual records.

[10] Project operations documentation: README.md, SECURITY.md, DATA-VERIFICATION.md, SETUP.md, DEPLOYMENT.md (accessed 16 October 2026).

ACKNOWLEDGEMENT

I/We thank [Project Guide Name] for guidance and feedback during this mini project, and the faculty and staff of [Department / College Name] for the opportunity and facilities to complete the work. I/We acknowledge the maintainers of React, TypeScript, Vite, Express, Prisma, SQLite, and other open-source tools used. Official government portals remain the authoritative source of current scheme and recruitment information.

Student name(s): _____

Date: _____